

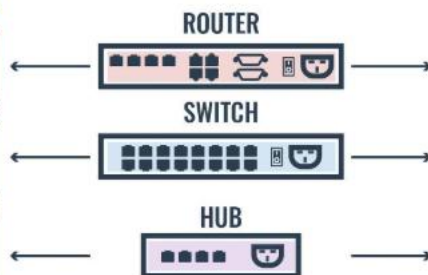
Teoría ¿Que es el protocolo HTTP y cómo funciona ?

Introducción

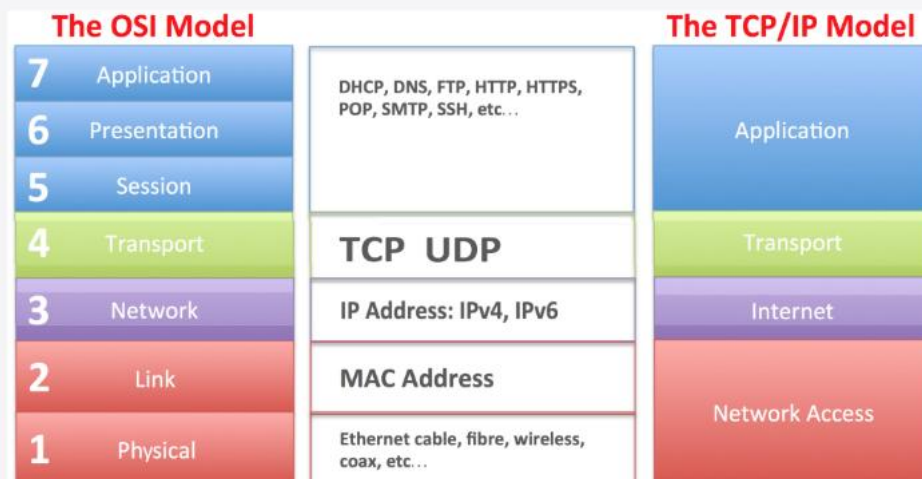
OSI REFERENCE MODEL



TCP/IP CONCEPTUAL LAYE



HTTP



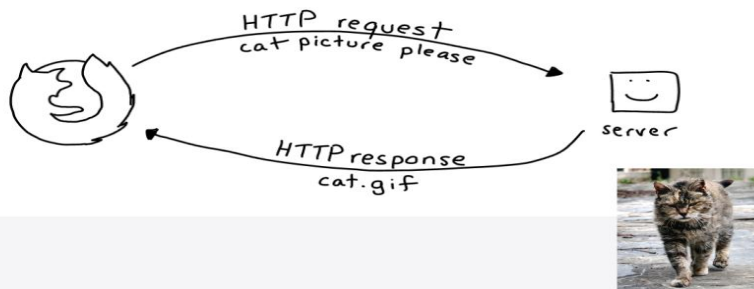
Mensajes Request y Response

HTTP: protocol



GET Request - HTTP

HTTP: GET Request



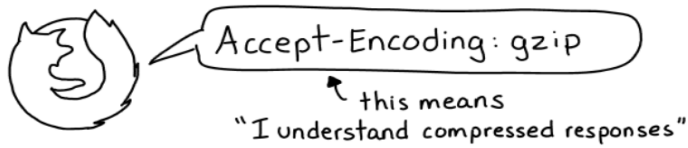
GET Request - HTTP

HTTP: Request / Response

request		response	
request line	{ GET / HTTP/1.1	status	{ HTTP/1.1 200 OK
headers	{ Host: examplecat.com	headers	{ Cache-Control: max-age=604800
	{ User-Agent: curl		{ Content-Type: text/html
	{ Accept: */*		{ Etag: "1541025663+ident"
	{ Server: ECS (nyb/1D0B)		
	{ Vary: Accept-Encoding		
			{ X-Cache: HIT
			{ Content-Length: 1270
		body	{ <!doctype html>
			{ <title>Example Cat</title>
			{ ...

GET Request / Response - HTTP

HTTP: 'header'



HTTP: 'header'

Describe the body:

Content-Type: image/png Content-Encoding: gzip
Content-Length: 12345 Content-Language: es-ES

Ask for a specific kind of response:

Accept: image/png Accept-Encoding: gzip
Range: bytes=1-10 Accept-Language: es-ES

Every Accept-
header has a
corresponding
Content- header

Manage caches:

ETag: "abc123" If-Modified-Since: 3 Aug 2019 13:00:00 GMT
If-None-Match: "abc123" Last-Modified: 3 Feb 2018 11:00:00 GMT
Vary: Accept-Encoding Expires: 27 Sep 2019 13:07:49 GMT
Cache-Control: public, max-age=300

Say where the request comes from:

User-Agent: curl Referer: https://examplecat.com

Cookies:

Set-Cookie: name=julia; HttpOnly (server → client)
Cookie: name=julia (client → server)

6 and more!

GET Request - HTTP

GET Request

HTTP: Request

method
(usually GET or POST) → GET /cat.png HTTP/1.1

resource being requested

HTTP version

domain being requested → Host: examplecat.com

headers { User-Agent: Mozilla...
Cookie:

HTTP Request

contiene... :

- *domain* (p.ej. examplecat.com)
- *resource* (p.ej.: /cat.png)
- *method* (p.ej.: GET, POST ...)
- *headers* (extra info. -> servers)

Hay un Request 'Body' opcional. GET Request no suele usarlo pero POST Request sí.

POST Request

HTTP: POST (JSON)

The diagram shows an HTTP POST request structure with handwritten annotations:

- method**: An arrow points to `POST /add_cat HTTP/1.1`.
- headers**: A bracket groups the following lines:
 - `Host: examplecat.com`
 - `Content-Type: application/json` (An arrow points to this line with the note "content type of body".)
 - `Content-Length: 20`
- request body**: An arrow points to the JSON object `{"name": "mr darcy"}` with the note "request body: the JSON we're sending to the server".

Métodos

HTTP: methods - 'CRUD'

Table 1. Supported HTTP methods

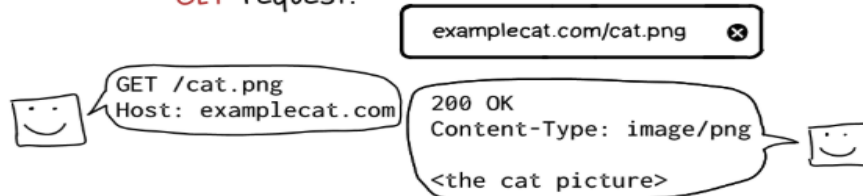
HTTP method	Description
POST	Creates a resource.
GET	Retrieves a resource.
PUT	Updates an existing resource. Does not create the resource if it does not exist.
DELETE	Deletes a resource.

Diferencias entre los métodos GET y POST

HTTP: GET vs POST

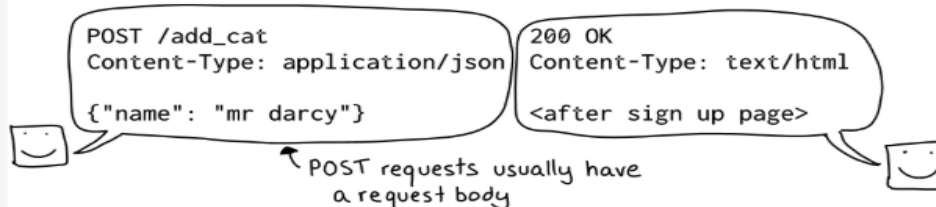
GET

When you type an URL into your browser, that's a GET request.



POST

When you hit submit on a form, that's (usually) a POST request.

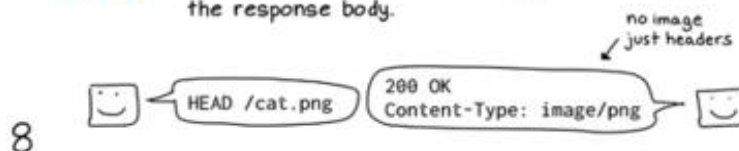


Otros métodos

HTTP: HEAD / OPTIONS / DEL...

HEAD

Returns the same result as GET, but without the response body.



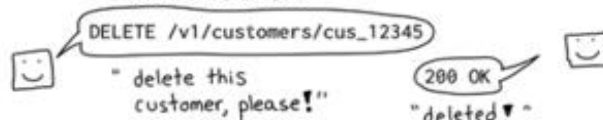
OPTIONS

OPTIONS is mostly used for CORS requests.

It also tells you which methods are available.

DELETE

Used in many APIs (like the Stripe API) to delete resources.



Clave API

HTTP: API Key

→ Identify yourself

Most HTTP APIs require a secret API Key so they know who you are.
Here's how that looks for the Twilio API:

```
curl
https://api.twilio.com/2010-04-01/Accounts/ACCOUNT_ID/Messages.json
-H "Content-Type: application/json"
-u ACCOUNT_ID:AUTH_TOKEN
-d '{
  "from": "+15141234567",
  "to": "+15141234567",
  "body": "a text message"
}'
```

*-u sends the username/password
in the Authorization header*

this sends a POST request

Respuesta HTTP

HTTP: anatomía de una respuesta

HTTP responses have:

- a status code (200 OK! 404 not found!)
- headers
- a body (HTML, an image, JSON, etc)

Here's the HTTP response from `examplecat.com/cat.txt`:

```
HTTP/1.1 200 OK
Accept-Ranges: bytes
Cache-Control: public, max-age=0
Content-Length: 33
Content-Type: text/plain; charset=UTF-8
Date: Mon, 09 Sep 2019 01:57:35 GMT
Etag: "ac5affa59f554a1440043537ae973790-ssl"
Strict-Transport-Security: max-age=31536000
Age: 0
Server: Netlify

\      /\
)  (  '
(  /  )
 \(_)_/

cat! 🐱
```

HTTP: anatomía de una respuesta

There are a few kinds of response headers:

→ when the resource was sent/modified:

Date: Mon, 09 Sep 2019 01:57:35 GMT
Last-Modified: 3 Feb 2017 13:00:00 GMT

→ about the response body:

Content-Language: en-US Content-Type: text/plain; charset=UTF-8
Content-Length: 33 Content-Encoding: gzip

→ caching:

ETag: "ac5affa..." Age: 255
Vary: Accept-Encoding Cache-Control: public, max-age=0

→ security:

X-Frame-Options: DENY Strict-Transport-Security: max-age=31536000
X-XSS-Protection: 1 Content-Security-Policy: default-src https:

→ and more:

Connection: keep-alive Accept-Ranges: bytes
Via: nginx
Set-Cookie: cat=darcy; HttpOnly; expires=27-Feb-2020 13:18:57 GMT;

Tipos de códigos de estado

HTTP: status codes

Every HTTP response has a ★status code★



There are 50ish status codes, and these are the most common ones in real life:

200 OK

} 2xx's mean
★ success ★

301 Moved Permanently

302 Found
temporary redirect

304 Not Modified
the client already has the latest
version, "redirect" to that

} 3xx's aren't
errors, just
redirects to
somewhere else

400 Bad Request

403 Forbidden
API key/OAuth/something needed

404 Not Found
we all know this one :)

429 Too Many Requests
you're being rate limited

} 4xx errors
generally mean
the client made
an invalid request

HTTP: status codes - 50X

500 Internal Server Error

the server code has an error

503 Service Unavailable

could mean nginx (or whatever proxy)

couldn't connect to the server

504 Gateway Timeout

the server was too slow to respond

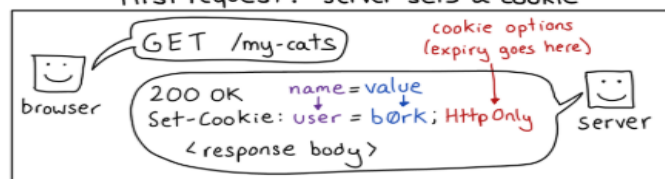
5xx errors
generally mean
something's wrong
with the server

Cookies

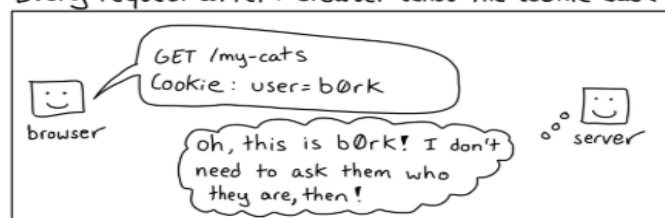
HTTP: cookies

They're set with the Set-Cookie response header, like this:

first request: server sets a cookie



Every request after: browser sends the cookie back



HTTP2

HTTP2: más rápido

before (HTTP/1.1)

→ request 1
response 1 ←
→ request 2
response 2 ←

after (HTTP/2)

→ request 1
→ request 2
out of order is OK { response 2 ←
response 1 ← } one TCP connection

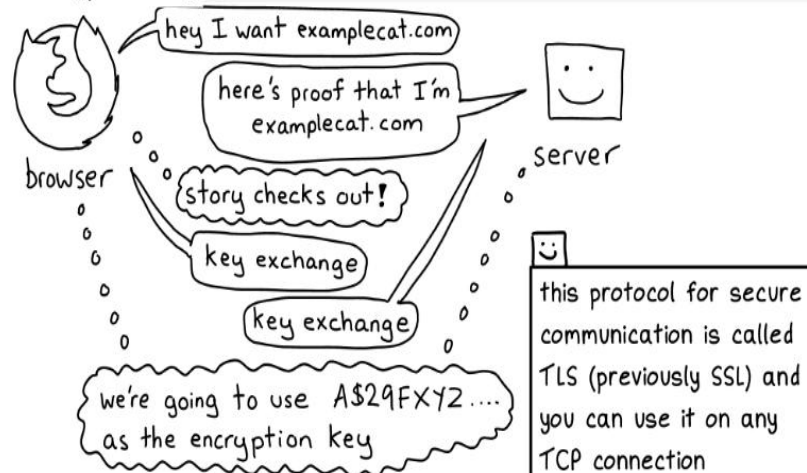
HTTPS

HTTPS: más seguro

HTTPS: HTTP + secure 

HTTPS: más seguro

① Negotiate an encryption key



② Write an HTTP request:

```
GET /cat.png HTTP/1.1
Host: examplecat.com
User-Agent: Mozilla/...
```

HTTPS: más seguro

③ Encrypt the HTTP request with AES & send it to examplecat.com:



④ Receive encrypted HTTP response:

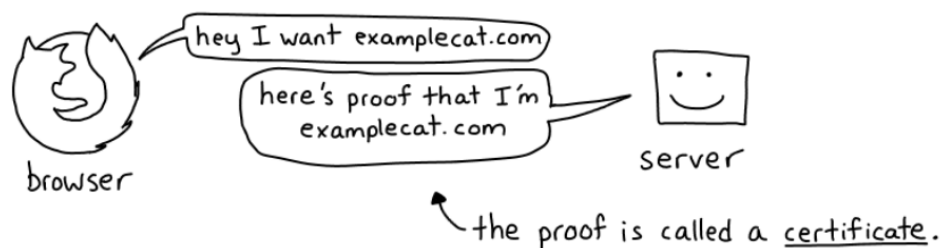


Encriptar una solicitud **HTTP request** con **AES** significa transformar los datos confidenciales (como el cuerpo o los parámetros) de una petición web en un galimatías ilegible utilizando el [Estándar de Cifrado Avanzado \(AES\)](#) antes de enviarla por la red.

Para qué sirve

- **Protección de carga útil (Payload):** Oculta información sensible dentro del cuerpo de un POST o PUT (contraseñas, datos bancarios o médicos) de miradas ajenas.
- **Seguridad en capas:** Aunque hoy en día se usa **HTTPS** (que cifra todo el canal de comunicación), algunos sistemas añaden AES a nivel de aplicación para proteger los datos incluso si el tráfico es interceptado o si se almacenan logs intermedios en texto plano.

HTTPS: +seguro - SSL/TLS



HTTPS: +seguro - SSL/TLS

